REST API EXAM – REFERENCE

server1.py · server2.js · router.py

CHANGE THIS	comment / tip	only if exam asks	copy as-is

1. KNOWN EXAM VARIANTS

Variant	Entity	Fields	Filter?	Extra	Router
v1 – Tickets	ticket	name, table, seats, paid(bool)	no	PUT update	Round Robin
v2 – Drinks	drink	name, category, price, available(bool)	no	PUT update	Ratio 1:3 JS:PY
v3 – Psychology	record	studentCode, issue, sessions, resolved(bool)	no	PUT update	Ratio 1:3 JS:PY
v4 – Animals	animal	species, breed, birthDate, registered(bool)	no	PUT update	Round Robin
v5 – Protocols	protocol	section_code, street, status, length_m …	?status=X	DELETE	Ratio 2:3 PY:JS

⚠ **PORTS NEVER CHANGE: Router=8000 Server1 Python=8001 Server2 JS=8002**

⚠ **EVERY response must include header: X-Server-ID: 1 or X-Server-ID: 2**

2. db.json

Create this file in the same folder before starting the servers. Change the field names to match your exam variant.

```
{
  "protocols": [   ◄ CHANGE THIS
  // ^^^ change to your entity name: animals / tickets / drinks / records
    {
      "id": 1,
      "field1": "value1",   ◄ CHANGE THIS
     // replace ALL these fields with the ones listed in your exam PDF
      "field2": "value2",   ◄ CHANGE THIS
      "field3": true   ◄ CHANGE THIS
    }
  ]
}
```

3. server1.py – Python Flask (port 8001)

What to do:

1. Copy the whole file
2. Change every yellow line (just the entity name protocols → yours)
3. Include the 🔵 PUT block if your exam asks for update
4. Include the 🔵 DELETE block if your exam asks for delete
5. If exam has no filter – delete the 3 filter lines inside get_all()

```python
from flask import Flask, jsonify, request
import json
app = Flask(__name__)

# ── HELPERS (never change) ──────────────────────────────────────
def load_data():
    with open("db.json", "r", encoding="utf-8") as f:
        return json.load(f)
def save_data(data):
    with open("db.json", "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)

# ── GET ALL ─────────────────────────────────────────────────────
@app.route("/protocols", methods=["GET"])    ◀ CHANGE THIS
def get_all():
    data = load_data()
    records = data["protocols"]    ◀ CHANGE THIS
    # --- filter block: delete these 3 lines if exam has no filter ---
    status_filter = request.args.get("status")
    if status_filter:
        records = [r for r in records if r.get("status") == status_filter]
    # ----------------------------------------------------------------
    response = jsonify({"protocols": records, "server": 1})    ◀ CHANGE THIS
    response.headers["X-Server-ID"] = "1"
    return response

# ── GET ONE ─────────────────────────────────────────────────────
@app.route("/protocols/<int:id>", methods=["GET"])    ◀ CHANGE THIS
def get_one(id):
    data = load_data()
    record = next((r for r in data["protocols"] if r["id"] == id), None)    ◀ CHANGE THIS
    if record is None:
        response = jsonify({"error": "Not found"})
        response.headers["X-Server-ID"] = "1"
        return response, 404
    response = jsonify(record)
    response.headers["X-Server-ID"] = "1"
    return response

# ── POST CREATE ─────────────────────────────────────────────────
@app.route("/protocols", methods=["POST"])    ◀ CHANGE THIS
def create():
    data = load_data()
    new_record = request.json
    new_record["id"] = max((r["id"] for r in data["protocols"]), default=0) + 1    ◀
CHANGE THIS
    data["protocols"].append(new_record)    ◀ CHANGE THIS
    save_data(data)
    response = jsonify({**new_record, "server": 1})
    response.headers["X-Server-ID"] = "1"
    return response, 201
```

```python
# ── PUT UPDATE  (only if exam asks for update) ──────────────────────
@app.route("/protocols/<int:id>", methods=["PUT"])  # change "protocols"
def update(id):
    data = load_data()
    record = next((r for r in data["protocols"] if r["id"] == id), None)
    if record is None:
        response = jsonify({"error": "Not found"})
        response.headers["X-Server-ID"] = "1"
        return response, 404
    updates = request.json
    updates.pop("id", None)    # never overwrite the id!
    record.update(updates)
    save_data(data)
    response = jsonify({**record, "server": 1})
    response.headers["X-Server-ID"] = "1"
    return response

# ── DELETE  (only if exam asks for delete) ──────────────────────────
@app.route("/protocols/<int:id>", methods=["DELETE"])  # change "protocols"
def delete(id):
    data = load_data()
    original_len = len(data["protocols"])  # change "protocols"
    data["protocols"] = [r for r in data["protocols"] if r["id"] != id]
    if len(data["protocols"]) == original_len:
        response = jsonify({"error": "Not found"})
        response.headers["X-Server-ID"] = "1"
        return response, 404
    save_data(data)
    response = app.response_class(status=204)
    response.headers["X-Server-ID"] = "1"
    return response

# ── START  (never change) ──────────────────────────────────────────
if __name__ == "__main__":
    app.run(port=8001, debug=True)
```

---

4. server2.js – Express.js (port 8002)

What to do:
Exactly the same as server1.py – just different language syntax.
Change every yellow line (entity name), include same optional blocks.

```javascript
const express = require("express");
const fs = require("fs");
const app = express();
app.use(express.json());

// ── HELPERS  (never change) ──────────────────────────────────────────
function loadData() { return JSON.parse(fs.readFileSync("db.json","utf-8")); }
function saveData(data) { fs.writeFileSync("db.json", JSON.stringify(data,null,2)); }

// ── GET ALL ──────────────────────────────────────────────────────────
app.get("/protocols", (req, res) => {   ◀ CHANGE THIS
    const data = loadData();
    let records = data.protocols;   ◀ CHANGE THIS
    // --- filter block: delete these 2 lines if exam has no filter ---
    const sf = req.query.status;
```

```
    if (sf) records = records.filter(r => r.status === sf);
    // ------------------------------------------------------------
    res.set("X-Server-ID", "2");
    res.status(200).json({ protocols: records, server: 2 });   ◄ CHANGE THIS
});

// ── GET ONE ──────────────────────────────────────────────────────
app.get("/protocols/:id", (req, res) => {   ◄ CHANGE THIS
    const data = loadData();
    // IMPORTANT: req.params.id is always a STRING - must parseInt!
    const id = parseInt(req.params.id);
    const record = data.protocols.find(r => r.id === id);   ◄ CHANGE THIS
    res.set("X-Server-ID", "2");
    if (!record) return res.status(404).json({ error: "Not found" });
    res.status(200).json(record);
});

// ── POST CREATE ──────────────────────────────────────────────────
app.post("/protocols", (req, res) => {   ◄ CHANGE THIS
    const data = loadData();
    const newRecord = req.body;
    const maxId = data.protocols.reduce((max,r) => Math.max(max,r.id), 0);   ◄ CHANGE
THIS
    newRecord.id = maxId + 1;
    data.protocols.push(newRecord);   ◄ CHANGE THIS
    saveData(data);
    res.set("X-Server-ID", "2");
    res.status(201).json({ ...newRecord, server: 2 });
});

// ── PUT UPDATE  (only if exam asks for update) ──────────────────
app.put("/protocols/:id", (req, res) => {   // change "protocols"
    const data = loadData();
    const id = parseInt(req.params.id);
    const index = data.protocols.findIndex(r => r.id === id);
    res.set("X-Server-ID", "2");
    if (index === -1) return res.status(404).json({ error: "Not found" });
    const updates = req.body;
    delete updates.id;   // never overwrite the id!
    data.protocols[index] = { ...data.protocols[index], ...updates };
    saveData(data);
    res.status(200).json({ ...data.protocols[index], server: 2 });
});

// ── DELETE  (only if exam asks for delete) ──────────────────────
app.delete("/protocols/:id", (req, res) => {   // change "protocols"
    const data = loadData();
    const id = parseInt(req.params.id);
    const index = data.protocols.findIndex(r => r.id === id);
    res.set("X-Server-ID", "2");
    if (index === -1) return res.status(404).json({ error: "Not found" });
    data.protocols.splice(index, 1);
    saveData(data);
    res.status(204).send();
});

// ── START  (never change) ────────────────────────────────────────
app.listen(8002, () => console.log("Server 2 running on port 8002"));
```

5. router.py – Load Balancer (port 8000)

What does the router actually do?

The client sends ALL requests to port 8000 (router) – never directly to 8001 or 8002.
The router looks at a counter and decides: send this to Python (8001) or JS (8002)?
It then copies the request there, gets the answer, and sends it back to the client.
You only need to change 2 things: 1) the SEQUENCE line 2) the entity name in routes.

Which SEQUENCE line to use?

Exam says …	Order of servers	SEQUENCE line
Round Robin	PY, JS, PY, JS …	["py","js"]
Ratio 2:3 PY:JS	PY, PY, JS, JS, JS …	["py","py","js","js","js"]
Ratio 1:3 JS:PY	JS, PY, PY, PY …	["js","py","py","py"]

```
from flask import Flask, request, Response
import requests
app = Flask(__name__)

# ── STEP 1: CHANGE THIS ONE LINE to match your exam ──────────────────
counter = 0
SEQUENCE = ["py","py","js","js","js"]   # ◄ CHANGE: see table above   ◄ CHANGE THIS

SERVERS = { "py": "http://localhost:8001", "js": "http://localhost:8002" }

def get_target():
    global counter
    target = SEQUENCE[counter % len(SEQUENCE)]
    # len(SEQUENCE) = 2 for Round Robin, 5 for 2:3, 4 for 1:3
    # counter % len cycles forever:  0,1,2,3,4,0,1,2,3,4 ...
    counter += 1
    return SERVERS[target]

# ── STEP 2: forward function  (never change this) ────────────────────
# takes the request, sends to chosen server, returns the response
def forward(path):
    url = get_target() + path
    resp = requests.request(
        method  = request.method,
        url     = url,
        params  = request.args,
        json    = request.get_json(silent=True),
        headers = {"Content-Type": "application/json"}
    )
    response = Response(resp.content, status=resp.status_code)
    if "Content-Type" in resp.headers:
        response.headers["Content-Type"] = resp.headers["Content-Type"]
    if "X-Server-ID" in resp.headers:
        response.headers["X-Server-ID"] = resp.headers["X-Server-ID"]
    return response

# ── STEP 3: routes - change "protocols" to your entity name ──────────
@app.route("/protocols", methods=["GET","POST"])   ◄ CHANGE THIS
# ^^^ add "PUT" to methods if exam has update without an id in URL
def collection():
    return forward("/protocols")   ◄ CHANGE THIS
```

```
@app.route("/protocols/<int:id>", methods=["GET","PUT","DELETE"])    ◄ CHANGE THIS
# ^^^ remove "DELETE" if no delete, remove "PUT" if no update
def single(id):
    return forward(f"/protocols/{id}")    ◄ CHANGE THIS

# ── START  (never change) ──────────────────────────────────────────
if __name__ == "__main__":
    app.run(port=8000, debug=True)
```

---

6. CHECKLIST BEFORE SUBMITTING

☐	Changed all "protocols" to your entity name in all 3 files
☐	Changed db.json field names to match your exam
☐	Added PUT block if exam asks for update
☐	Added DELETE block if exam asks for delete
☐	Removed filter lines if exam has no ?filter
☐	Picked the correct SEQUENCE line in router.py
☐	X-Server-ID header is on EVERY response ← already in the code above
☐	POST generates its own id and ignores incoming id ← already in code
☐	DELETE returns 204 not 200 ← already in code
☐	Ports are 8000 / 8001 / 8002 – NOT changed

Good luck!